

2023

تَلْخِصٌ شَامِلٌ لِأَسَاسِيَّاتِ بِيئَةِ الْعَمَلِ لِ: Laravel دَلِيلُ الْمُبْتَدِئِينَ.

مُلَخَّصٌ - LARAVEL

إعداد م. محمد عادل البحيسي
مطور ومدرّب- ويب

هو إطار عمل مفتوح المصدر لتطوير تطبيقات الويب بلغة (PHP). تم تطويره بواسطة Taylor Otwell وأصدر لأول مرة في عام 2011. يهدف Laravel إلى تسهيل عملية تطوير تطبيقات الويب وتوفير هيكلية وأدوات قوية للمطورين.

يتميز Laravel بالعديد من الميزات المفيدة والقوية، مثل نظام التوجيه (Routing system) لإدارة طرق الوصول إلى التطبيق، ونظام قواعد البيانات (Database system) الذي يدعم عدة قواعد بيانات مثل MySQL و PostgreSQL وغيرها، ونظام ORM (Object-Relational Mapping) الذي يسهل التعامل مع قواعد البيانات بشكل كائني، ونظام التشفير والمصادقة والترخيص (Encryption, Authentication, and Authorization) لحماية التطبيقات، وغيرها الكثير من الميزات المفيدة.

باستخدام Laravel، يمكن للمطورين بناء تطبيقات الويب بشكل سريع وفعال، وتنظيم الشفرة بشكل جيد باستخدام هيكلية MVC (Model-View-Controller)، وتوفير تجربة مطور مرنة وممتعة.

تم اعتماد Laravel بشكل واسع في صناعة تطوير الويب ويحظى بشعبية كبيرة بين المطورين نظرًا لسهولة الاستخدام والقدرة على التوسع والتكامل مع مكتبات وأدوات أخرى. ولسهولة رفعها على **الاستضافة**.

ما هي الاستضافة :

الاستضافة (Hosting) هي خدمة توفرها شركات الاستضافة (Hosting Providers) للأفراد والشركات لنشر وتشغيل مواقع الويب وتطبيقات الإنترنت على الإنترنت. تتيح خدمة الاستضافة للمواقع أن تكون متاحة وقابلة للوصول عبر الإنترنت.

تقوم شركات الاستضافة بتخصيص مساحة على **الخوادم (Server)** التي تعمل على الإنترنت لتخزين الملفات وقواعد البيانات والمحتوى الخاص بالمواقع. كما توفر أيضًا موارد الحوسبة اللازمة مثل المعالجة والذاكرة ونطاق النطاق الترددي (Bandwidth) لتمكين الوصول إلى الموقع وتشغيله. ومن أشهر شركات الاستضافة هي **استضافة ال Hostinger**

ما هو Server :

- هو جهاز كمبيوتر أو نظام يوفر الخدمات والموارد للأجهزة الأخرى المتصلة به عبر شبكة الإنترنت ويحمل IP عام. وال **iP (Internet Protocol)** هو بروتوكول يستخدم لتحديد عنوان كل جهاز متصل بشبكة الإنترنت. يُعتبر عنوان IP معرفًا **فريدًا** لكل جهاز في الشبكة يتكون من سلسلة من الأرقام العشرية المفصولة بنقاط، مثل 192.168.0.

يعمل الخادم على استقبال طلبات من العملاء (clients) وتقديم الخدمات المطلوبة لهم. يمكن أن يشمل الخدمات التي يوفرها الخادم استضافة المواقع الإلكترونية، وتخزين البيانات، وتشغيل التطبيقات، وتوفير خدمات البريد الإلكتروني، وإدارة قواعد البيانات، والتواصل والتوجيه بين الأجهزة المتصلة.

يعمل الخادم عادة بنظام تشغيل مخصص له مثل **Windows Server** أو **Linux**، ويستخدم برامج خادم مثل **Apache** أو **Nginx** لمعالجة وتوجيه طلبات العملاء وتقديم الخدمات المطلوبة. يتم استخدام الخوادم في البيئات المؤسسية والشبكات المحلية وعلى الإنترنت لتوفير الخدمات للمستخدمين والعملاء.

ما هو Domain :

النطاق أو الدومين (Domain) هو عنوان ويب يستخدم لتمييز موقع الويب على الإنترنت. وهو عبارة عن سلسلة من الأحرف والأرقام ورموز النقطة، يستخدم لتحديد مكان وجود الموقع على السيرفر (هو عبارة عن الغلاف الخاص ب IP الي تم حجزه في السيرفر). مثال عليه [https:// tmooh-dev.com](https://tmooh-dev.com)

ما هو HTTP :

هي اختصار لـ "**Hypertext Transfer Protocol**" وتعني بالعربية "بروتوكول نقل النص المتشعب". إنه بروتوكول يستخدم في توصيل المعلومات عبر الإنترنت والشبكات الحاسوبية. يعتبر HTTP أساسيًا في عملية تصفح الويب، حيث يسمح للمتصفح بالتواصل مع الخوادم وطلب المحتوى المطلوب من صفحات الويب. يستخدم HTTP لنقل البيانات بين المتصفح والخادم بطريقة متسلسلة، حيث يتم إرسال طلبات من المتصفح إلى الخادم، ويتم الرد عليها بالمعلومات المطلوبة. وتتم نقطة الاتصال به على السيرفر على **Port =80** .

ما هو HTTPS :

هو اختصار لـ "**Hypertext Transfer Protocol Secure**" ، وهو بروتوكول آمن لنقل المعلومات عبر الإنترنت. يعتبر HTTPS الإصدار المشفّر والمحمي من HTTP، تختلف HTTPS عن HTTP في الطريقة التي يتم بها نقل البيانات بين المستخدم والخادم. في حالة استخدام HTTPS ، تتم إضافة طبقة تشفير تسمى **SSL/ (Secure Sockets)** لحماية البيانات أثناء النقل. تتم عملية التشفير عن طريق تشفير البيانات وفك تشفيرها باستخدام شهادة رقمية تصدر للموقع وتحقق صحة الاتصال. وتتم نقطة الاتصال به على السيرفر على **Port =443** .

ما هو Port :

ال **Port** (المنفذ) هو رقم يستخدم لتحديد القناة المحددة لتبادل البيانات بين جهازين عبر شبكة الإنترنت أو شبكة الاتصال الأخرى. يتم تعيين رقم المنفذ لكل بروتوكول معين (هي نقطة الاتصال على السيرفر).

ما هو SubDomain :

هو جزء من اسم النطاق العام (**Domain**) الذي يتم تعريفه قبل النطاق الرئيسي. يتم فصل النطاق الفرعي عن النطاق الرئيسي باستخدام نقطة (.) في العنوان. يستخدم النطاق الفرعي في العديد من السيناريوهات، مثل توجيه حركة المرور إلى أجزاء محددة من الموقع، أو إنشاء عناوين البريد الإلكتروني المخصصة، أو تنظيم الخدمات والتطبيقات داخل الموقع، وغيرها من الأغراض. **ولكن يجيب ان تكون الاستضافه الخاصه بك تدعم ال SubDomain**

مثال عليه <https://tech.tmooh-dev.com>

يشير مصطلح **URI (Uniform Resource Identifier)** إلى تعبير يستخدم في تحديد موارد الويب بشكل فريد وعالمي. يمكن اعتباره هوية فريدة تميز موارد الويب مثل الصفحات الويب والصور ومقاطع الفيديو والوثائق والملفات الأخرى المتاحة على الإنترنت. ويتكون **URI** من اثنين من الأجزاء الرئيسية وهما:

1. **URL (Uniform Resource Locator)**: يُستخدم للإشارة إلى عنوان موقع محدد على الإنترنت. يتضمن عنوان **URL** بروتوكول الاتصال مثل **HTTP** أو **HTTPS** واسم النطاق مثال عليه <https://tech.tmooh-dev.com>

2. **URN (Uniform Resource Name)**: يُستخدم لتعيين اسم دائم للمورد بغض النظر عن موقعه الفعلي. على عكس **URL** الذي يشير إلى موقع محدد، يتم استخدام **URN** لتحديد المورد بناءً على اسمه المستقل. مثال عليه <https://tech.tmooh-dev.com/ar/blogs>

وتعتبر المعادله الخاصه بل **URI** هي عبارة عن:

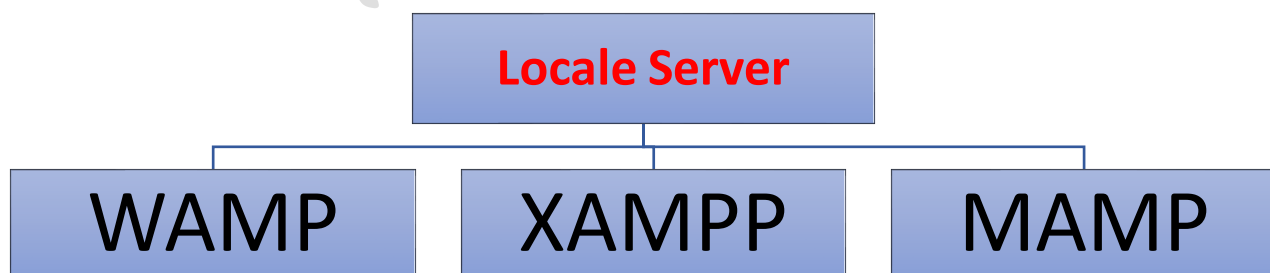
$$URL + URN = URI$$

مثال عليه: <https://tech.tmooh-dev.com/ar/blogs>

ولنفترض أنك لا تمتلك سيرفر ما الحل؟ الحل هو أنك تعمل بيئته وهميه على جهاز الكمبيوتر الخاص بك وتسمي هذه البيئه بشئ اسمه **اللوكل سيرفر (Locale Server)**: فما هو اللوكل سيرفر وما هي أشهر البرامج الخاص به؟

هي بيئات تطوير محلية (Local Development Environments) تستخدم لتشغيل تطبيقات الويب وقواعد البيانات على جهاز الكمبيوتر الشخصي، تسمح هذه البيئات للمطورين بتشغيل واختبار مواقعهم على الجهاز المحلي دون الحاجة إلى خادم ويب حقيقي. تشتمل على تطبيقات الخادم مثل **Apache** ونظم إدارة قواعد البيانات مثل **MySQL** أو **MariaDB** ولغات البرمجة الخاصة بالويب مثل **PHP**.

من أشهر البرامج الخاصه Locale Server هم:



1. **WAMP** وتعني **Windows + Apache + MySQL + PHP** تعتبر **WAMP** بيئة تطوير محلية مصممة لنظام التشغيل **Windows**. تشمل على العناصر الأساسية التالية:

- **Windows** : نظام التشغيل.
- **Apache** : خادم الويب.
- **MySQL** : نظام إدارة قواعد البيانات.
- **PHP** : لغة برمجة السكريبت المستخدمة لتطوير تطبيقات الويب.

1. **XAMPP** وتعني **Cross-platform + Apache + MySQL + PHP + Perl** تعتبر **XAMPP** بيئة تطوير محلية متعددة المنصات ويمكن استخدامها في أنظمة التشغيل المختلفة مثل **Linux** و **Windows** و **Mac**. تشمل على العناصر الأساسية التالية:

- **Cross-platform** : تعمل على عدة أنظمة التشغيل.
- **Apache** : خادم الويب.
- **MySQL** : نظام إدارة قواعد البيانات.
- **PHP** : لغة برمجة السكريبت المستخدمة لتطوير تطبيقات الويب.
- **Perl** : لغة برمجة قوية تستخدم لتطوير تطبيقات الويب والسكريبتات.

MAMP هو اختصار لـ **Mac + Apache + MySQL + PHP** يعتبر **MAMP** بيئة تطوير محلية مصممة خصيصًا لنظام التشغيل **macOS**. تشمل على العناصر الأساسية التالية:

- **Mac** : نظام التشغيل، **macOS**.
- **Apache** : خادم الويب.
- **MySQL** : نظام إدارة قواعد البيانات.
- **PHP** : لغة برمجة السكريبت المستخدمة لتطوير تطبيقات الويب.

MAMP يوفر بيئة محلية كاملة لتطوير تطبيقات الويب على نظام **macOS**. يتم تثبيته كتطبيق واحد يحتوي على جميع المكونات الضرورية لتشغيل الويب، ويتم تكوينه بسهولة واستخدامه في بيئة تطوير محلية قبل نشر التطبيقات على الخوادم الحقيقية.

1. **بيئة التطوير المحلية (Local Server Environment)**: هذه الخطوة تتضمن إعداد بيئة تشغيل محلية على جهاز الكمبيوتر الخاص بك لتشغيل تطبيقات الويب. هنا سوف نستخدم بيئة العمل الوهمية وهي ال **WAMP** ورابط التنزيل هو <https://www.wampserver.com>.
2. تنزيل ملف **vc redistrib**: هو برنامج يحتوي على المكتبات المشتركة التي تحتاجها بيئة العمل **WAMP** ليتم تشغيله بشكل صحيح. يجب تنزيل وتثبيت الإصدار المناسب لنظام التشغيل الخاص بك ورابط التنزيل هو <https://www.techpowerup.com/download/visual-c-redistributable-runtime-package-all-in-one>.
3. تنزيل **Composer**: هو أداة إدارة الاعتمادات في **PHP**. يستخدم لتثبيت وإدارة المكتبات والإضافات اللازمة لمشروعك. يمكنك تنزيله وتثبيته من الموقع الرسمي. ورابط التنزيل هو <https://getcomposer.org>.
4. تنزيل **Git**: هو عبارة عن الوسيط أو الشريك ما بين ال **Composer** وال **Laravel** على ال **Github** يستخدم لتتبع التغييرات في مشروعك وتتبع التغييرات. يمكنك تعقب التعديلات التي تم إجراؤها على الملفات وتحديد من أجرى التغييرات ومتى تم إجراؤه. ورابط التنزيل هو <https://git-scm.com/downloads>.
5. تنزيل **Visual Studio Code (VSCode)**: هو محرر نصوص مفتوح المصدر وخفيف الوزن. يتميز بميزات قوية لتحرير الشفرة والتكامل مع أدوات التطوير المختلفة. يمكنك تنزيله وتثبيته من موقعه الرسمي. ورابط التنزيل هو <https://code.visualstudio.com/download>.

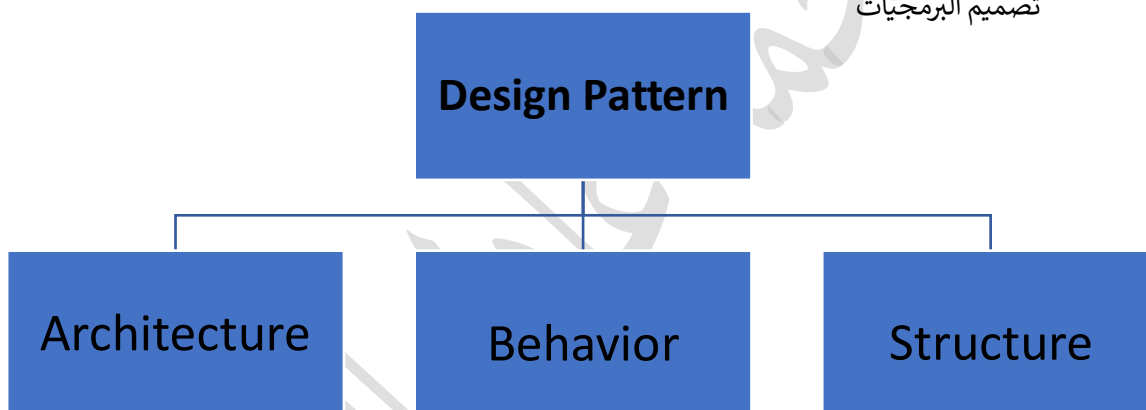
بعض ال Extension اللازمة لتهيئة برنامج الفيچوال ستوديو كود :

1. **PHP Debug**: توفر دعمًا لتصحيح الشفرة المصدرية في لغة **PHP** داخل **Visual Studio Code**. يمكنك استخدامها لتنفيذ تعليمات مفصلة ومراقبة قيم المتغيرات أثناء تشغيل تطبيق **Laravel**.
2. **PHP Intelephense**: هو ملحق يوفر **IntelliSense** قويًا وتحسينات في تكملة الشفرة وتوصيات الرمز للغة **PHP** في **Visual Studio Code**. يساعدك في الكتابة السريعة والدقيقة وزيادة الإنتاجية أثناء تطوير تطبيقات **Laravel**.
3. **Laravel Blade Snippets**: يقدم مجموعة واسعة من التعبيرات المختصرة (**Snippets**) للتعامل مع قوالب **Blade** في **Laravel**. يسهل عليك كتابة الشفرة وتكرار الأكواد الشائعة في قوالب **Blade**.
4. **Laravel Artisan**: هو أداة سطر أوامر تأتي مع إطار عمل **Laravel**. توفر أوامر **Artisan** وظائف قوية لإدارة التطبيق مثل إنشاء وتنفيذ قاعدة البيانات وإنشاء نماذج وتوليد المفاهيم الأساسية للتطبيق.
5. **Laravel Snippets**: يوفر ملحق **Laravel Snippets** مجموعة من التعبيرات المختصرة للشفرة في **Visual Studio Code**. يمكنك استخدامها لتسهيل الكتابة والوصول السريع إلى أكواد **Laravel** الشائعة.
6. **Laravel goto view**: يوفر هذا الملحق ميزة الانتقال السريع إلى ملفات العرض (**Views**) في **Laravel**. يمكنك ببساطة فتح الملفات المرتبطة بأعمال العرض دون الحاجة إلى البحث اليدوي.
7. **Laravel Extra Intellisense**: هو ملحق يقدم تحسينات إضافية لخدمة **IntelliSense** في **Visual Studio Code** عند العمل مع **Laravel**. يوفر مزيدًا من التوصيات للشفرة وتوفيرًا أكبر للوقت أثناء التطوير.

8. **DotENV**: يوفر ملحق DotENV دعمًا لتنسيق ملفات **env** في **Laravel** يساعدك في التحقق من المتغيرات المحددة في ملف **env** وتحميلها في تطبيق **Laravel** الخاص بك.
9. **vscode-icons**: هو ملحق يقدم أيقونات توضيحية جميلة ومخصصة للملفات والمجلدات في **Visual Studio Code** يمكن أن يكون لها فائدة في تنظيم وتصفح ملفات مشروع **Laravel** الخاص بك.
10. **SFTP**: يعد (Secure File Transfer Protocol) أداة تمكنك من الاتصال بخادم **SFTP** ونقل الملفات بطريقة آمنة. يمكنك استخدامها للوصول إلى ملفات تطبيق **Laravel** الموجودة على الخادم البعيد وتحريرها وتحميلها من خلال **Visual Studio Code**.

اللقاء الثاني

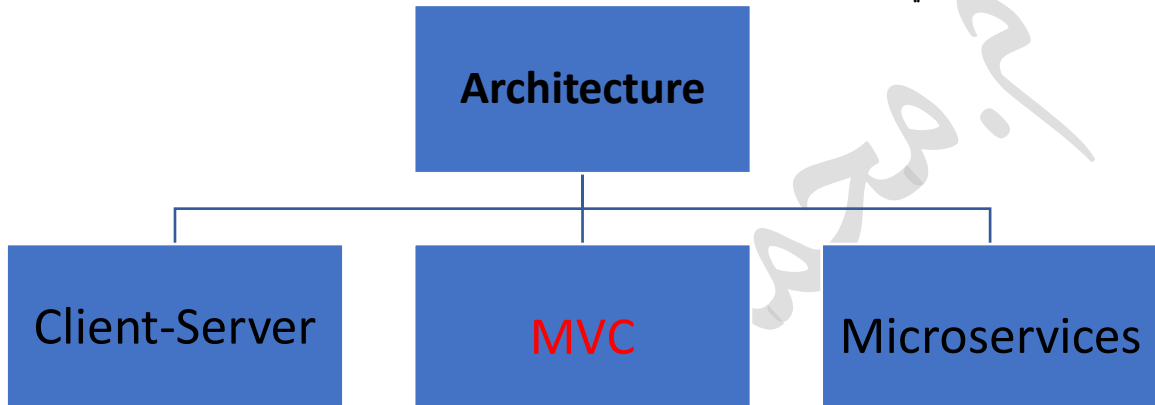
Design Pattern: أسلوب التصميم يشير إلى مفهوم عام يستخدم في تطوير البرمجيات لحل مشكلات التصميم المتكررة وتوفير حلول مألوفة ومجربة. يعتبر أسلوب التصميم نمطًا معممًا لحل مشكلة معينة في تصميم البرمجيات



1. **Architecture** (الهندسة المعمارية): تشير إلى تصميم الهيكل العام والتنظيم الكبير لنظام البرمجيات. يركز على تحديد المكونات الرئيسية للنظام والعلاقات بينها وكيفية تنظيمها وتفاعلها لتحقيق أهداف النظام بشكل فعال. وهي التي تم بناء ال **Laravel** عليها.
2. **Behavior** (السلوك): يتعلق بتصميم سلوك وتفاعل المكونات في نظام البرمجيات. يهدف إلى تحديد كيفية استجابة المكونات للأحداث والتفاعلات المختلفة وتنظيم سلوكها بطريقة قابلة للتوسيع وإدارة.
3. **Structure** (الهيكل): يركز على تصميم هيكل البرنامج وتنظيم المكونات الداخلية والعلاقات بينها. يتضمن تحديد كيفية تنظيم الكود والوحدات والتفاعلات بينها لتحقيق هيكلية صحيحة وسهولة الصيانة وإعادة الاستخدام.

عندما تشير إلى "**Architecture**" في سياق تطوير البرمجيات، فإنه يشير إلى تصميم الهندسة المعمارية للنظام أو التطبيق. تصميم الهندسة المعمارية يتعامل مع تحديد الهيكل العام للنظام وتنظيمه وتحديد تفاعلات المكونات والموديولات المستخدمة والقواعد والمبادئ التوجيهية التي يجب اتباعها لبناء النظام بشكل فعال ومستدام.

عند الحديث عن أنواع التصميم المعماري، هناك عدة أنماط شائعة في تطوير البرمجيات. ومن بين هذه الأنماط التصميمية الشائعة هي:



(**Client-Server**): يستند إلى توزيع المسؤوليات بين العميل (Client) والخادم (Server)، حيث يتم تنفيذ جزء من التطبيق على العميل وجزء آخر على الخادم، ويتم تبادل البيانات بينهما.

(**Model-View-Controller (MVC)**): يتم فيه تقسيم التطبيق إلى ثلاثة مكونات رئيسية، وهي النموذج (**Model**) والعرض (**View**) والتحكم (**Controller**). يعمل النموذج على التعامل مع المنطق والبيانات، في حين يعمل العرض على عرض البيانات للمستخدم، والتحكم يدير التفاعل بين النموذج والعرض.

Microservices: يعتمد على تقسيم التطبيق إلى مجموعة من الخدمات الصغيرة والمستقلة، وكل خدمة تعمل بشكل مستقل وتتفاعل مع الخدمات الأخرى عبر واجهات برمجة التطبيق (APIs).

يستخدم **Laravel** نمط التصميم المعماري (**MVC (Model-View-Controller)**) ويتم تقسيم التطبيق إلى ثلاثة أقسام:

- النموذج (**Model**): يتعامل مع إدارة البيانات والمنطق المرتبطة بها.
- العرض (**View**): يتعامل مع عرض المحتوى وتفاعل المستخدم معه.
- التحكم (**Controller**): يدير تنسيق واستجابة النموذج والعرض.

يشير ال **View** الجزء الخاص بالعرض ويعتبر العرض واجهة التي يتفاعل معها المستخدم ويتم عرض المعلومات له.

سابقا قبل ظهور ال **Laravel** كان المطورين يستلمون من مصممين ال **FrontEnd** ملفات ذات امتداد **viewName.html** ويتم تحويلهم الي **viewName.php** والعمل بل **php Native** ولكن عند ظهور **Laravel**

أصبح الأمر مختلف تم تحويل امتداد صفحات العرض من **viewName.php** الى **viewName.blade.php**. فما هو ال **Blade**.

Blade: هو عبارة عن أمتداد خاص بملف العرض في ال **Laravel** هذا الإمتداد يعطي القدرة لمطورين ال **Laravel** بكتابه كود يطلق على هذا الكود اسم **Code Directives** هذا الكود مخصص في **Laravel** في صفحات **blade** يبدأ دائما بعلامة ال **@**.

في سياق بناء تطبيقات الويب باستخدام **Laravel**، يمكن أن يُفهم ال **Model** على أنه عبارة عن ملف ذات امتداد **PHP** تمثل جدولًا في **قاعدة البيانات**. يتم استخدام ال **Model** للتعامل مع البيانات المخزنة في **قاعدة البيانات** وتنفيذ العمليات المرتبطة بها.

عند إنشاء **Model** في **Laravel**، يتم تعريف الخصائص (**properties**) والعلاقات (**relationships**) التي تتبع هيكل جدول قاعدة البيانات المرتبط بها. يتم استخدام ال **Model** للوصول إلى البيانات من جداول قاعدة البيانات وتنفيذ العمليات المختلفة مثل الاستعلام (**querying**) والإدخال (**inserting**) والتحديث (**updating**) والحذف (**deleting**).

فما هي قاعدة البيانات؟

قاعدة البيانات (**Database**) هي مجموعة من البيانات المنظمة والمخزنة بشكل مرتب في هيكل معين. تستخدم قواعد البيانات لتخزين وتنظيم البيانات بحيث يسهل الوصول إليها وإدارتها. تستخدم في تطبيقات البرمجيات للحفاظ على البيانات والسماح بالبحث والتحليل والتلاعب بها بطريقة فعالة ومنظمة.

هناك عدة أنواع مختلفة من قواعد البيانات من أشهرها:

- **MySQL**
- **SQLite**
- **MariaDB**

وغيرهم من الأنواع الأخرى وجميع هذه الأنواع تم بنائهم بلغه برمجيه من نوع **SQL**

- (**Structured Query Language**) الي هي بللغه العربيه **لغه الاستعلام الهيكلية**.

قاعدة البيانات: هي مكان تخزين المعلومات المنظمة بطريقة هيكليّة. تتكون قاعدة البيانات من مجموعة من الجداول، ولكل جدول يتم تعيين اسم فريد له. عادةً ما يُفضل أن يكون اسم الجدول في صيغة الجمع، وأن يبدأ الحرف الأول من اسم الجدول بحرف صغير. في حالة تكوين اسم الجدول من كلمتين أو أكثر، يفضل استخدام الشرطة السفلية (_) بين كل كلمة والكلمة التالية. والكلمة الأخيرة في اسم الجدول عادة ما تكون الجمع للمعلومات التي تتم تخزينها في الجدول. جميع هذه التوصيات تساهم في تسمية الجداول بشكل واضح ومفهوم للمطورين والمستخدمين.

وكل جدول يتكون من عدد من **الأعمدة** التي تحدد نوع البيانات والخصائص لكل عمود.

من بين أنواع البيانات الشائعة في قاعدة البيانات:

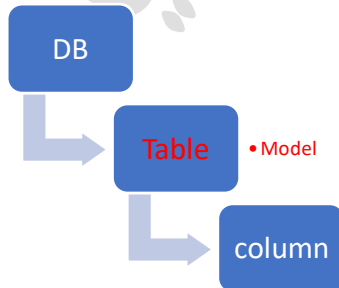
- النص: (String) يستخدم لتخزين النصوص والمحارف مثل الأسماء والعناوين.
- العدد الصحيح: (Integer) يستخدم لتخزين الأعداد الصحيحة بدون أعشار.
- العدد عائِم: (Float) يستخدم لتخزين الأعداد العشرية والأعداد ذات الأعشار العائمة.
- التاريخ والوقت: (Date and Time) يستخدم لتخزين التواريخ والأوقات.

بالإضافة إلى ذلك، يمكن تطبيق بعض الخصائص على الأعمدة مثل:

- القيمة الفريدة: (Unique) تضمن أن كل قيمة في العمود فريدة وغير مكررة.
- القيمة الفارغة: (Null) تسمح بإدخال قيمة فارغة أو غير معرفة في العمود.
- القيمة الافتراضية: (Default Value) تعين قيمة افتراضية للعمود عندما لا يتم توفير قيمة جديدة.
- القيود: (Constraints) تضمن قواعد الصحة والتحقق للقيم المخزنة في العمود مثل القيد الأدنى والقيد الأقصى.

تلك بعض الأمثلة على أنواع البيانات والخصائص المشتركة في قواعد البيانات. تختلف الأنواع والخصائص المتاحة بين أنظمة إدارة قواعد البيانات المختلفة.

يتم توليد صفوف من خلال تقاطع الأعمدة. يمثل كل صف مجموعة من البيانات المرتبطة معًا ويتم تمثيل كل صف بسجل واحد في الجدول.



و الموديل يعتبر طريقه من 3 طرق سنتطرق لهم لاحقا

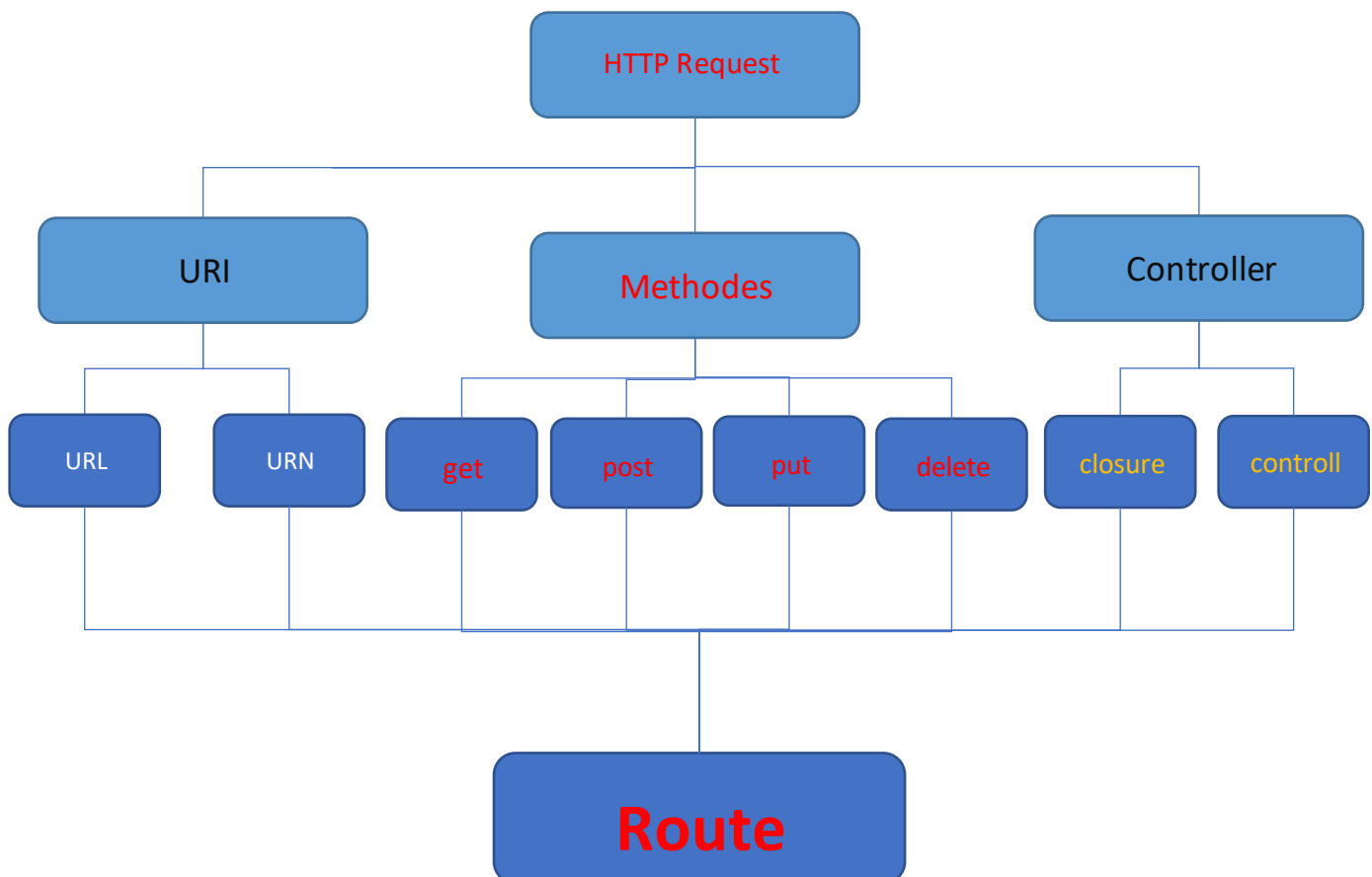
ال **Controller** هو ملف أو جزء من الكود يعمل كوسيط بين ال **View** وال **Model** في تطبيق الويب. يقوم ال **Controller** بالتحكم في العمليات والتفاعلات بين ال **View** وال **Model** ، وذلك لعرض البيانات وإدارة العمليات المتعلقة بالتطبيق.

بمعنى آخر، يعتبر ال **Controller** مسؤولاً عن استقبال طلبات المستخدم من ال **View** وتحليلها، ثم التفاعل مع ال **Model** للحصول على البيانات المطلوبة وإجراء العمليات اللازمة. بعد ذلك، يتولى ال **Controller** إعداد وتنسيق البيانات لعرضها في ال **View** بالشكل المناسب.

يمكننا اعتبار ال **Controller** بمثابة الجسر الذي يربط بين ال **View** وال **Model** ، حيث يسهل عليهما التواصل والتعاون لتحقيق أهداف التطبيق. يعتبر ال **Controller** جزءاً هاماً في نمط التصميم **MVC** ، ويساعد في جعل التطبيق منظماً وسهل الصيانة وفهم الشفرة المصدرية.

سيطول شرح **Controller** لاحقاً ولكن هذه نبذة تعريفية عنه.

عندما يقوم المستخدم بزيارة صفحة ويب أو يقوم بإجراء إجراء معين على موقع الويب، يتم إرسال طلب من المستخدم إلى الخادم. يحتوي طلب على معلومات محددة حول العملية التي يرغب المستخدم في تنفيذها ويسمى الطلب هذا باسم **HTTP Request**:

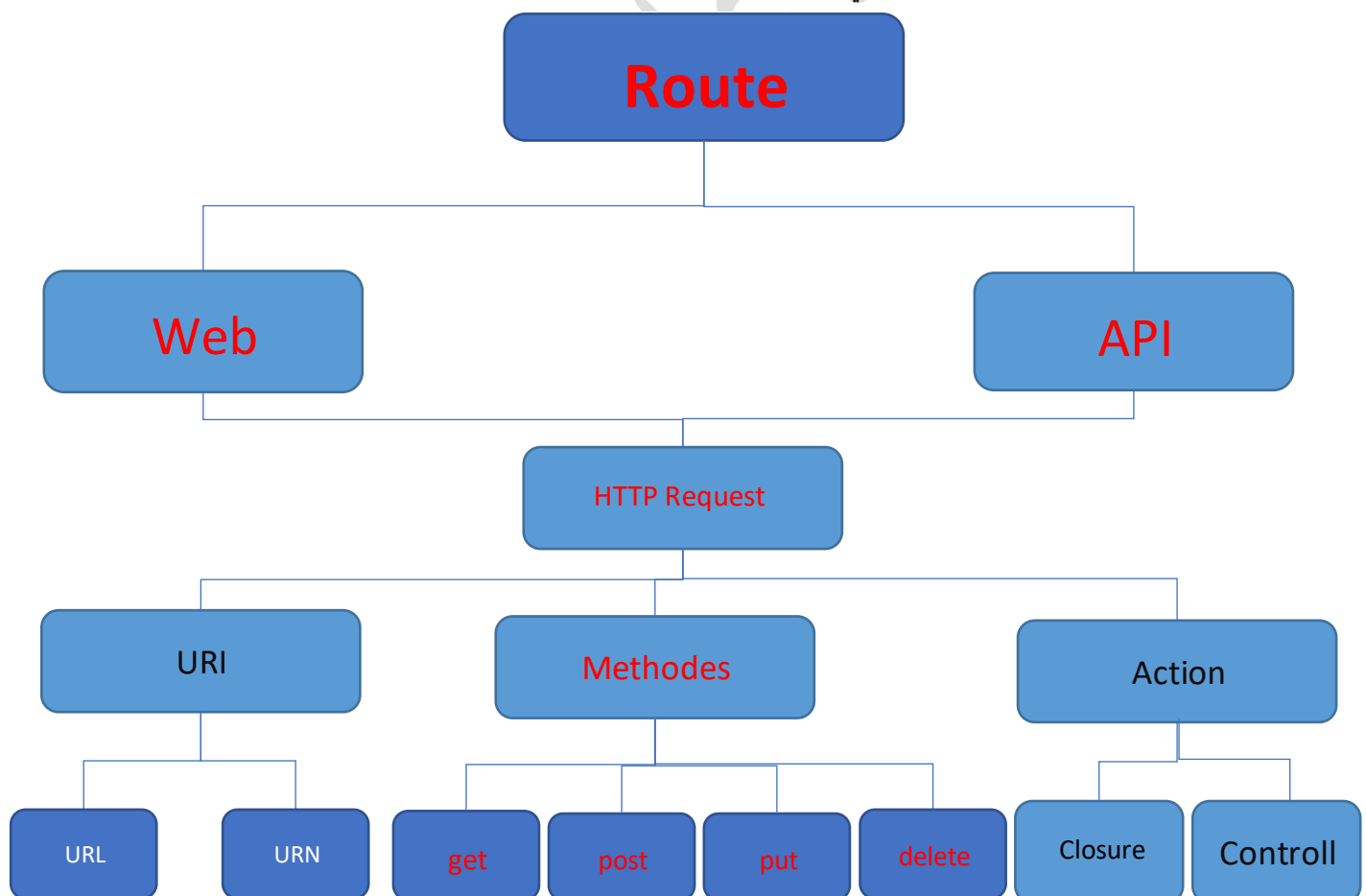


Route في Laravel هو آلية تستخدم لتحديد الروابط والتوجيهات بين عناوين URL والوظائف المرتبطة بها في تطبيقنا. يتألف كل Route من ثلاثة عناصر رئيسية، الأول هو **URI** وهو عنوان فريد يمثل المورد المستهدف، الثاني هو **Methods** وهو نوع طريقة الوصول المسموحة مثل **GET** أو **POST** أو **PUT** أو **DELETE**، والثالث هو **Controller** وهو الوحدة المسؤولة عن تنفيذ الوظائف المرتبطة بال Route.

يتم استخدام ال URI لتحديد المسار الدقيق الذي يجب أن يتم الوصول إليه في التطبيق. بمجرد الوصول إلى هذا المسار، يتم التحقق من طريقة الوصول المستخدمة والتحقق مما إذا كانت تطابق Route المحددة. بعد ذلك، يتم توجيه الطلب إلى ال Controller المحدد لتنفيذ الوظائف المطلوبة، مثل استرجاع البيانات أو تحديثها أو حذفها.

تستخدم ال Methods في Laravel لتحديد أنواع طرق الوصول المسموحة لكل Route. يمكن أن تكون هذه الأنواع مثل **GET** لاسترجاع البيانات، **POST** لإرسال بيانات جديدة، **PUT** لتحديث بيانات موجودة، و **DELETE** لحذف بيانات.

باستخدام ال Controller، يتم تعيين وتحديد الوحدة المسؤولة عن تنفيذ وإدارة الوظائف المرتبطة بال Route. يمكن أن تكون ال Controller عبارة عن صف PHP يحتوي على طرق ووظائف للتعامل مع البيانات وإعادة الاستجابة المناسبة. وهناك نوعان من ال Route في Laravel

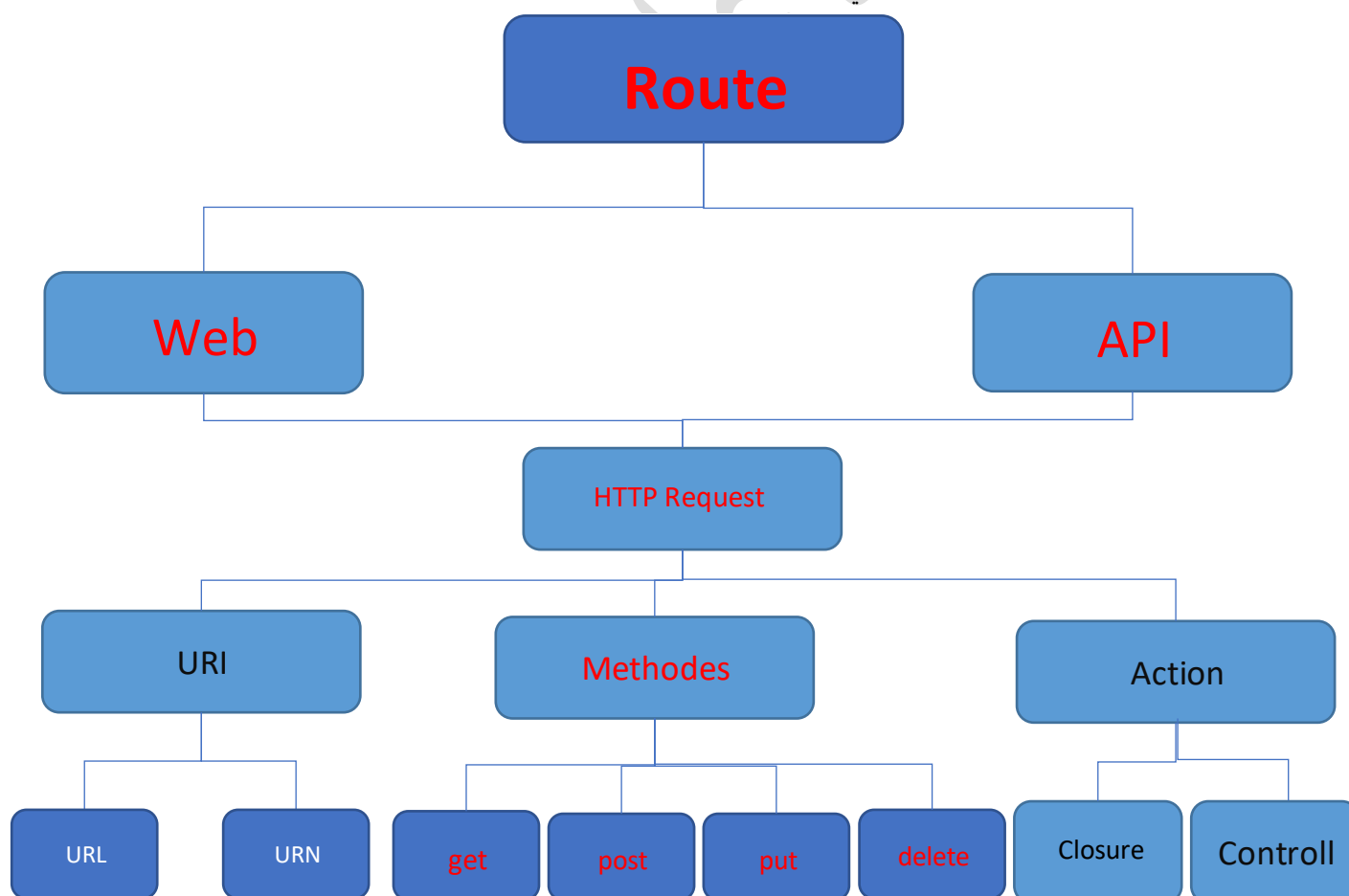


Route في Laravel هو آلية تستخدم لتحديد الروابط والتوجيهات بين عناوين URL والوظائف المرتبطة بها في تطبيقنا. يتألف كل Route من ثلاثة عناصر رئيسية، الأول هو **URI** وهو عنوان فريد يمثل المورد المستهدف، الثاني هو **Methods** وهو نوع طريقة الوصول المسموحة مثل **GET** أو **POST** أو **PUT** أو **DELETE**، والثالث هو **Controller** وهو الوحدة المسؤولة عن تنفيذ الوظائف المرتبطة بال Route.

يتم استخدام ال URI لتحديد المسار الدقيق الذي يجب أن يتم الوصول إليه في التطبيق. بمجرد الوصول إلى هذا المسار، يتم التحقق من طريقة الوصول المستخدمة والتحقق مما إذا كانت تطابق Route المحددة. بعد ذلك، يتم توجيه الطلب إلى ال Controller المحدد لتنفيذ الوظائف المطلوبة، مثل استرجاع البيانات أو تحديثها أو حذفها.

تستخدم ال Methods في Laravel لتحديد أنواع طرق الوصول المسموحة لكل Route. يمكن أن تكون هذه الأنواع مثل **GET** لاسترجاع البيانات، **POST** لإرسال بيانات جديدة، **PUT** لتحديث بيانات موجودة، و **DELETE** لحذف بيانات.

باستخدام ال Controller، يتم تعيين وتحديد الوحدة المسؤولة عن تنفيذ وإدارة الوظائف المرتبطة بال Route. يمكن أن تكون ال Controller عبارة عن صف PHP يحتوي على طرق ووظائف للتعامل مع البيانات وإعادة الاستجابة المناسبة. وهناك نوعان من ال Route في Laravel

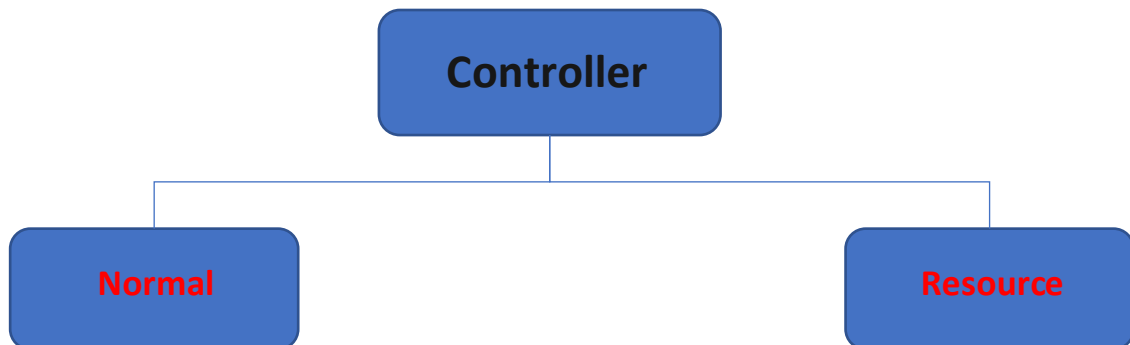


1. **Controller العادي (Normal Controller)** يُعرف أيضًا باسم "عادي" أو "بدون موارد (non-resourceful)" ، وهو المُتحكم الأساسي في Laravel. يُستخدم لتعريف طرق (methods) المُتحكم الخاصة بالتفاعلات المختلفة في التطبيق. يمكن تعريف أي طريقة تتناسب مع احتياجات التطبيق، مثل "index" لعرض قائمة العناصر أو "store" لحفظ بيانات جديدة .

```
php artisan make:controller UserController
```

2. **Controller الموارد (Resource Controller)** يُستخدم لتعريف مُتحكم يتمثل في طرق معينة تتعامل مع عمليات الاسترجاع والإنشاء والتحديث والحذف (CRUD) لنماذج (Models) محددة. يقدم Laravel طرقًا قياسية لمعالجة هذه العمليات في المُتحكمات الموارد، مثل "index" و "create" و "store" و "edit" و "update" و "destroy". يُمكنك استخدام الأمر التالي لإنشاء مُتحكم موارد للمستخدمين (Users)

```
php artisan make:controller UserController --resource
```



ملف الكنترولر في Laravel يحتوي عادةً على الكود الخاص بالطرق (Methods) التي يستجيب بها المُتحكم لطلبات المستخدم ويتعامل مع البيانات والمنطق المتعلقة بالمشروع. لكل طريقة تعني بوحدة من العمليات الأساسية (مثل عرض البيانات، إنشاء سجل جديد، تحديث سجل موجود، حذف سجل) يُعرف في الكنترولر طريقة مقابلة.

بداية الملف، يجب تعريف المساحة الاسمية (Namespace) المطابقة لموقع الملف في التطبيق. مثلاً، إذا كان الملف ينتمي للمجلد "App\Http\Controllers" ، فيجب أن يكون المساحة الاسمية كالتالي:

```
namespace App\Http\Controllers;
```

ثم يتم استيراد **الكلاسات** الضرورية التي قد تحتاجها في الكنترولر، مثل النماذج (Models) أو الكلاسات المساعدة (Helper Classes).
بعد ذلك، يُعرف الكلاس نفسه. اسم الكلاس يجب أن يكون مطابقًا لاسم الملف (باستثناء الامتداد) وينتهي بـ "Controller". على سبيل المثال، إذا كان اسم الملف هو "UserController.php"، يجب أن يكون اسم الكلاس "UserController".

```
class UserController extends Controller
{
    // المنطق المرتبط بها (Methods) الطرق
}
```

ما هو ال class :

في برمجة الكائنات (Object-Oriented Programming)، ال class هي هيكلية أساسية تعرف سلوك وخصائص الكائنات. وتعتبر ال class نموذجًا يستخدم لإنشاء كائنات فردية (instances) من نوع معين.

النقاط المهمة التي يجب اتباعها عند بناء ال class في PHP على النحو التالي:

1. اسم ال class يجب أن يبدأ بحرف كبير للتمييز بينه وبين المتغيرات والدوال الأخرى.
 2. إذا كان اسم ال class يتكون من أكثر من كلمة، يجب أن يتم استخدام حرف كبير للبداية في كل كلمة.
 3. يُمنع استخدام أي علامات خاصة في اسم ال class، بما في ذلك العلامة "\$"، لأنها محجوزة في لغة PHP.
 4. يُمنع استخدام علامة "_" في اسم ال class، ويُفضل تجنب استخدام أي علامات خاصة تمامًا.
- عند اتباع هذه النقاط، يسهل فهم وقراءة ال class وتفاعله مع بقية الكود. يساعد الالتزام بتلك القواعد في الحفاظ على تنظيم ووضوح الكود وتجنب الاشتباكات مع الأسماء المحجوزة في لغة PHP.

في البرمجة الكائنية، ال **constructor** (أو **الدالة البنائية**) هي دالة خاصة في ال class يتم استدعاؤها تلقائيًا عند إنشاء كائن جديد من ال class. يتم استخدام ال **constructor** لإعداد الكائن وتهيئته بقيم افتراضية أو تنفيذ أي مهام أخرى قبل استخدام الكائن.

فائدة ال constructor هي عبارة عن الدالة المستخدمة في إنشاء **Object** من ال Class وال **Object** => عنصر

لتعريف ال constructor في PHP، يتم استخدام الدالة **__construct()** داخل ال class. عند إنشاء كائن من ال class، يتم استدعاء ال constructor تلقائيًا ويتم تنفيذ الكود الموجود فيه.

بإمكانك إنشاء أكثر من نسخة (كائن) من ال class. يمكنك فعل ذلك ببساطة عن طريق استدعاء ال constructor وإنشاء كائنات جديدة من ال class بواسطة الكلمة المفتاحية **"new"**.

فيما يلي مثال يوضح كيفية إنشاء عدة نسخ من ال class "User":

```
class User
{
    public function __construct()
    {
        //
    }
}

$user = new User();
```

هذه الكود يقوم بتعريف الـ class "User" وإنشاء كائن واحد منه.

تم تعريف الـ class "User" وتم إضافة الـ **__construct ()** constructor الذي لا يحتوي على أي عمليات. ثم تم إنشاء كائن واحد من الـ class "User" باستخدام الكلمة المفتاحية "new" وتخزينه في المتغير "\$user". وفي حالة تنفيذ هذا الكود، تم إنشاء كائن واحد من الـ class "User" وتعيينه للمتغير "\$user". ولكن لم يتم تنفيذ أي عمليات أخرى داخل الكائن أو استخدامه بأي شكل آخر.

يمكنك استخدام المتغير "\$user" للعمل مع الكائن الذي تم إنشاؤه، مثل استدعاء الوظائف أو تعيين قيم للسمات، ولكن يجب عليك توسيع الـ class "User" وإضافة المزيد من الوظائف والسمات حسب احتياجاتك.

هل يسمح بتعريف متغيرات داخل الكلاس؟

نعم، يُسمح بتعريف متغيرات داخل الـ class في لغة PHP. يمكنك تعريف المتغيرات داخل الـ class واستخدامها في أعضاء الـ class الأخرى مثل الدوال والوظائف. ولكن قبل أن نعرف المتغيرات يجب أن نضع قبل كل متغير معامل وصول أو (Access Modifier).

Access modifiers (معاملات الوصول) هي عبارة عن كلمات تستخدم لتحديد مدى إمكانية الوصول إلى الخصائص والأساليب والثوابت داخل الـ class في لغات البرمجة مثل PHP.

هناك ثلاثة أنواع رئيسية لمعاملات الوصول في PHP:

1. **Public:** يتيح الوصول إلى الخاصية أو الوظيفة من أي مكان، سواء داخل الـ class نفسها أو من الكود الخارجي.
2. **Protected:** يتيح الوصول إلى الخاصية أو الوظيفة داخل الـ class نفسها ولأي class يرث منها، ولكن لا يمكن الوصول إليها من الكود الخارجي.
3. **Private:** يقيد الوصول إلى الخاصية أو الوظيفة بالـ class نفسها فقط، ولا يمكن الوصول إليها من الـ class الخارجية أو الكود الخارجي.


```
class MyClass {
    public $variable1;
    private $variable2 = "Value";
    protected $variable3;
}
```

بالإضافة إلى المعاملات الوصول، هناك أيضًا ما يسمى بـ **Non-access modifiers** (تعديلات غير الوصول) في لغات البرمجة مثل PHP. تعديلات غير الوصول تستخدم لتعديل سلوك وسمات الأعضاء داخل الـ class بدلاً من مستوى الوصول إليها.

تعديلات غير الوصول تتضمن ما يلي:

1. **Final**: يُضاف قبل تعريف الـ class أو الوظيفة لمنع تمديدتها أو التعديل عليها من قبل الـ class الفرعية (subclasses) أو الوظائف الأخرى. في حالة الـ class، لا يمكن تمديدتها. في حالة الوظيفة، لا يمكن استبدالها. (override).

2. **Static**: يُضاف قبل تعريف الـ property أو الوظيفة لجعلها تنتمي إلى الـ class نفسه بدلاً من الكائنات المنشأة من الـ class. يعني أن الخاصية أو الوظيفة يتم مشاركتها بين جميع الكائنات المنشأة من الـ class.

في حالة استدعاء متغير من نوع Non-access modifiers، يتم استخدام الـ (::) للوصول إلى المتغير من الكلاس نفسه،

مثال على **Non-access modifiers**:

Static: يسمح بالوصول إلى المتغير مباشرة من خلال الكلاس نفسه دون الحاجة إلى إنشاء كائن جديد من الكلاس والتعديل عليه

```
class Counter {
    public static $count = 0;
}

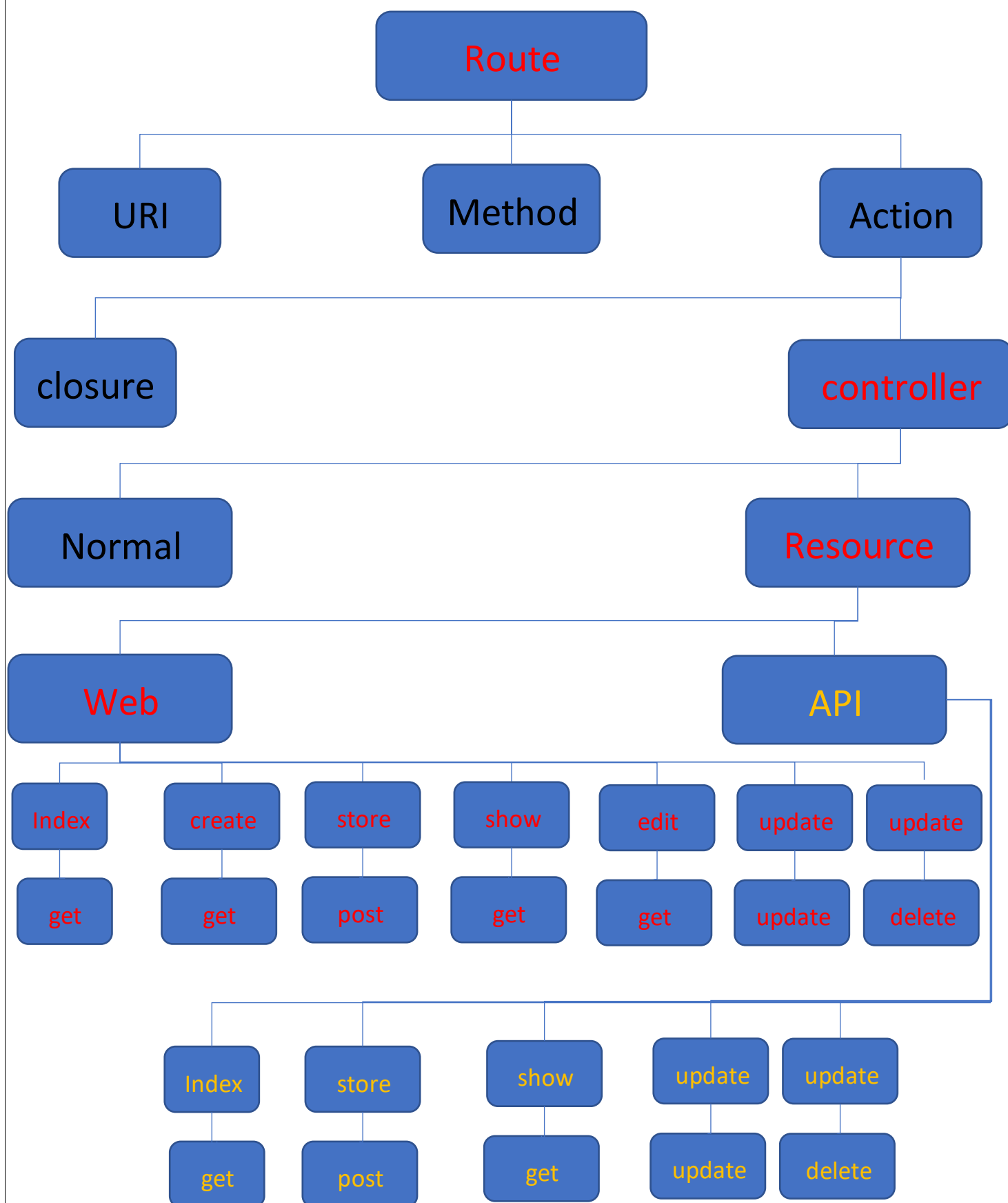
echo Counter::$count; // Output: 0
```

Final: هي قيمه ثابتة ولا تتغير يتم تعريفها باشي اسمو **const**

```
class MathUtils {
    public final PI = 3.14; // اسمو const لا يمكن تعريف الثابت داخل الكلاس بل صيغته باشي اسمو
    const PI = 3.14;
}

echo MathUtils::PI;
```

الآن وبعد ما تم شرح كل ما يتعلق بمحتوى ال **Controller** : سننتقل الآن الى تفاصيل أنواع الكنترولر:



بعد ما تحدثنا عن طريقه الوصول الي البيانات في ال **DataBase** وقلنا انه يمكننا الوصول اليها عن طريق ربط **Controller** مع ال **Model** وقلنا انها تعتبر طريقه من ثلاثة طرق وذلك مشروح في سلايد رقم (9):

هناك ثلاثة أنواع رئيسية للاستعلامات في Laravel على قواعد البيانات (**Model (Eloquent)** و **Query Builder** و **SQL** النقي. سنقوم بشرح موجزًا لكل منها ومثال على كيفية استخدامها:

-Model (Eloquent) : تسمح بالتفاعل مع قواعد البيانات باستخدام نموذج موجود. يعتبر **Model (Eloquent)** أحد أكثر الميزات شيوعًا واستخدامًا في Laravel. يتيح لك (**Model (Eloquent)**) تمثيل الجداول في قاعدة البيانات باستخدام نماذج PHP والقيام بعمليات قاعدة البيانات بطريقة سلسلة وبديهية.

مثال على استخدام: **Model (Eloquent)**

لنفترض أن لدينا جدولًا يسمى "users" يحتوي على حقول "id" و "name" و "email". يمكننا استخدام **Model (Eloquent)** لاسترداد جميع المستخدمين من الجدول ببساطة كما يلي:

```
use App\Models\User;

$users = User::all();

foreach ($users as $user) {
    echo $user->name;
}
```

في هذا المثال، نستخدم الوظيفة **all ()** لاسترداد جميع السجلات في جدول المستخدمين، ثم نقوم بعرض أسماء المستخدمين.

Query Builder: Query Builder هو طريقة أخرى لبناء استعلامات قواعد البيانات في Laravel باستخدام تعبيرات PHP. يسمح **Query Builder** لك ببناء استعلامات قاعدة البيانات بشكل برمجي ومرونة عالية.

مثال على استخدام: **Query Builder**

لنفترض أننا نريد استرداد المستخدمين الذين يملكون عنوان بريد إلكتروني محدد، يمكننا استخدام **Query Builder** كما يلي:

```
$users = DB::table('users')
    ->where('email', 'example@example.com')
    ->get();

foreach ($users as $user) {
    echo $user->name;
}
```

في هذا المثال، نستخدم `DB::table('users')` للتحديد أننا نقوم ببناء استعلام على جدول المستخدمين، ثم نستخدم `where()` لتحديد الشرط أن البريد الإلكتروني يساوي 'example@example.com'، وأخيرًا نستخدم `get()` لاسترداد السجلات المطابقة.

SQL- Laravel تسمح أيضًا بكتابة استعلامات **SQL** النقية مباشرة إذا كان هناك حاجة لذلك. يمكنك استخدام الوظيفة `DB::select()` لتنفيذ استعلامات **SQL** واسترداد النتائج ككائنات قابلة للاستخدام.

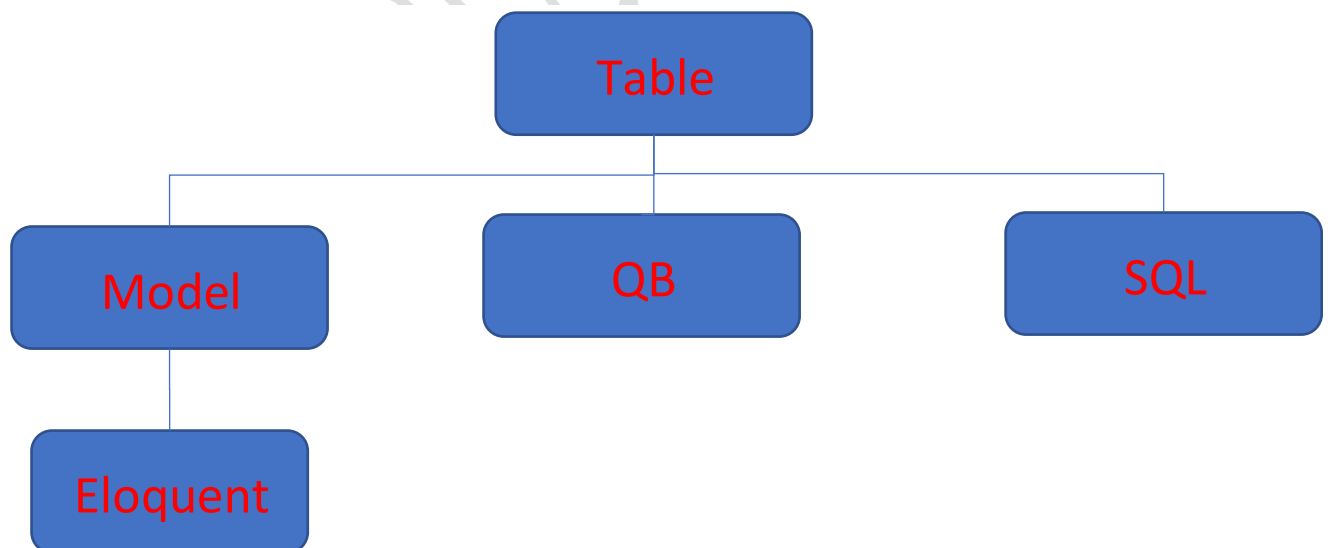
مثال على استخدام SQL :

```
$users = DB::select('SELECT * FROM users');

foreach ($users as $user) {
    echo $user->name;
}
```

في هذا المثال، نستخدم `DB::select()` لتنفيذ استعلام **SQL** بسيط يسترد جميع السجلات في جدول المستخدمين، ثم نقوم بعرض أسماء المستخدمين.

هذا المخطط يلخص أنواع الاستعلام في Laravel :



1. كل جدول في ال **BD** يقابله مودل واحد فقط.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    protected $table = 'users';
    protected $fillable = ['name', 'email', 'password'];
}
```

عند قيامنا بإنشاء نماذج (Models) لتمثيل الجداول في قاعدة البيانات. في هذا المثال، لدينا نموذج يسمى User يمثل جدول المستخدمين في قاعدة البيانات. يتم تعريف النموذج باستخدام كلمة المفتاح class مع اسم النموذج User وكلمة المفتاح extends و Model.

فما هي ال **extends**: هي عبارة عن علاقة توريث تحصل ما بين شيء يسمى ال الأب يعني (Parent) او **class** أو **base**) وما بين الإبن يعني (child أو derived أو sub) والوراثة تحصل ما بين كلاس وكلاس وكل كلاس يرث من كلاس واحد فقط وليش شرط اجباري ان يحصل وراثه بين الكلاسات

في المثال المرفق أعلاه ال **Model**: هو الصنف الأساسي (الأب) الذي يتم تمديد النماذج منه في لارافيل. يوفر Model العديد من الوظائف والسمات التي تجعل من السهل التعامل مع قاعدة البيانات. على سبيل المثال، يوفر Model وظائف لاستعلام وتحديث البيانات، وإجراء عمليات مشتركة مثل البحث والفرز، وإعداد العلاقات بين الجداول وغيرها الكثير.

باستخدام extends Model، يرث النموذج User جميع الوظائف والسمات المتوفرة في Model، مما يسهل علينا الوصول والتعامل مع قاعدة البيانات. نحن نحدد أيضًا اسم الجدول الذي يرتبط به النموذج باستخدام الخاصية `protected $table = 'users'`. يمكننا أيضًا تحديد الحقول التي يمكن تعديلها في النموذج باستخدام الخاصية `$fillable` وتحديد العلاقات بين الجداول في حال وجودها.

جميع هذه المعلومات ستشرح بشكل أوسع في القاءات بإذن الله

```

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    // بقية التعريفات والعلاقات في النموذج
}

```

كلمة المفتاح **use** الموضوعه داخل الكلاس في لغة البرمجة PHP تستخدم لاستيراد الـ **Traits** أو الـ **Classes** أو الـ **Namespaces** في ملف PHP الحالي. تقوم بتعريف مكان الـ **Trait** أو الـ **Class** أو الـ **Namespace** بحيث يمكن استخدامها بداخل الكود الموجود في الملف الحالي.

والـ **Trait** / عبارة عن ملف برمجي شبيهه الـ **class** في خصائصه ما عدا:

- لا يورث
- ولا يحتوي على **construct** وعندما نتحدث عن الـ **construct** يعني انه لا يمكننا انشاء **Object** ولا يوجد عدد معين لاضافة الـ **Trati** الي الكلاس.

3. يمكننا من خلال المودل التحكم في جميع عناصر الجدول:
4. يمكننا أيضا التحكم في اتصال المودل مع الجدول المناسب.

- لإنشاء كترولر من نوع **Normal**:

```
php artisan make:controller UserController
```

- لإنشاء كترولر من نوع **Attribute Binding**:

Attribute Binding يتيح لك ربط قيم الطرق بواسطة السمات (Attributes) للنموذج. بمعنى آخر، يمكنك استخدام قيم الطرق كقيم للحقول في النموذج.

على سبيل المثال، إذا كان لديك طريق لعرض معلومات المستخدم على الرابط "users/{id}/"، يمكنك استخدام **Attribute Binding** لربط قيمة المعرف (id) بحقل في نموذج المستخدم (User Model) واستخدامها لاستعادة بيانات المستخدم المناسبة.

```
php artisan make:controller UserController --resource
```

عد تنفيذ الأمر، سيتم إنشاء ملف الكترولر "UserController.php" في المسار "app/Http/Controllers".

يمكنك فتح الملف وتعديل الدوال والمنطق الخاص بك داخله، وسيتم إنشاء الدوال الأساسية لعمليات **CRUD** (القراءة والإدخال والتحديث والحذف) تلقائياً.

- لإنشاء كترولر من نوع **Model Binding**:

Model Binding يتيح لك ربط قيم الطرق مباشرة بنموذج (**Model**) في Laravel. يتم تحويل القيمة المحددة في الطريق إلى كائن من النموذج المقابل تلقائياً.

على سبيل المثال، إذا كان لديك طريق لعرض معلومات المستخدم على الرابط "users/{user}/"، يمكنك استخدام **Model Binding** لربط قيمة المعرف (user) مباشرة بنموذج المستخدم (User Model).

```
php artisan make:controller UserController --model=User
```

واستخدامها لاستعادة بيانات المستخدم المناسبة.

ومن هنا نستنتج خلاصة أنواع الكترولر سواء كان Web ام Api من نوع **Resource**

Controller Resource

Model Binding

Attribute Binding